

Lambda Calculus

Your Favorite Language

Probably has lots of features:

- Assignment ($x \leftarrow x + 1$)
- Booleans, integers, characters, strings, ...
- Conditionals
- Loops
- ~~return, break, continue~~
- Functions
- Recursion
- References / pointers
- Objects and classes
- Inheritance
- ...

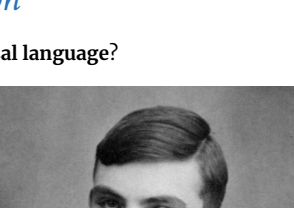
Which ones can we do without?

What is the smallest universal language?

What is computable?

Before 1930s

Informal notion of an **effectively calculable function**:



can be **computed** by a human with pen and paper, following an **algorithm**

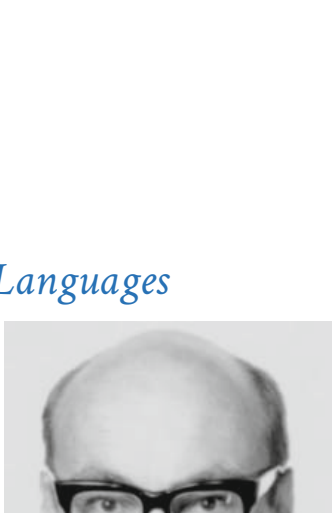
1936: Formalization

What is the **smallest universal language**?



Alan Turing

UK (Cambridge)



Alonzo Church

Alonzo Church

The Next 700 Languages



Peter Landin

Whatever the next 700 languages turn out to be, they will surely be variants of **lambda calculus**.

Peter Landin, 1966

The Lambda Calculus

Has one feature:

- Functions

No, really

- Assignment ($x \leftarrow x + 1$)
- Booleans, integers, characters, strings, ...
- Conditionals
- Loops
- ~~return, break, continue~~
- **Functions**
- **Recursion**
- References / pointers
- Objects and classes
- Inheritance
- Reflection

More precisely, only thing you can do is:

- Define a function
- Call a function

Describing a Programming Language

- **Syntax**: what do programs **look like**?
- **Semantics**: what do programs **mean**?
 - Operational semantics: how do programs execute step-by-step?

$e ::= x, y, z, \dots$
 $\mid (\lambda x \rightarrow e)$
 $\mid (e_1 e_2)$

$x, y, z, \dots$
 $\text{function}(x) \{ \text{return } e \}$
 $e_1(e_2)$

Syntax: What Programs Look Like

$e ::= x$ -- variable 'x'
 $\mid (\lambda x \rightarrow e)$ -- function that takes a parameter 'x' and returns 'e'
 $\mid (e_1 e_2)$ -- call (function) 'e1' with argument 'e2'

Programs are **expressions** e (also called λ -terms) of one of three kinds:

- Variable
 - x, y, z
- **Abstraction** (aka **nameless** function definition)
 - $(\lambda x \rightarrow e)$
 - x is the **formal parameter**, e is the **body**
 - "for any x compute e "
- **Application** (aka function call)
 - $(e_1 e_2)$
 - e_1 is the function, e_2 is the argument
 - in your favorite language: $e_1(e_2)$

(Here each of e, e_1, e_2 can itself be a variable, abstraction, or application)

Some Simple Expr

x
 bob
 cat

$(bob\ cat)$

$(bob\ (cat\ hog))$

$(\lambda x \rightarrow bob)\ (\lambda y \rightarrow cat)$

Examples

$(\lambda x \rightarrow x)$ -- The identity function (id) that returns its input

$(\lambda x \rightarrow (\lambda y \rightarrow y))$ -- A function that returns (id)

$(\lambda f \rightarrow (f (\lambda x \rightarrow x)))$ -- A function that applies its argument to id

QUIZ

Which of the following terms are syntactically **incorrect**?

- A. $(\lambda (x \rightarrow x) \rightarrow y)$ **NO** (ie NOT in λ -calc)
- B. $(\lambda x \rightarrow (x\ x))$ **YES**
- C. $(\lambda x \rightarrow (x\ (y\ x)))$ **YES**

D. A and C

E. all of the above

Examples

$(\lambda x \rightarrow x)$ -- The identity function (id) that returns its input

$(\lambda x \rightarrow (\lambda y \rightarrow y))$ -- A function that returns (id)

$(\lambda f \rightarrow (f (\lambda x \rightarrow x)))$ -- A function that applies its argument to id

How do I define a function with two arguments?

- e.g. a function that takes x and y and returns y ?

$\lambda x \rightarrow \lambda (x\ y)$ **not valid** λ -calc

$\lambda x \rightarrow (\lambda y \rightarrow y)$

$e ::= x$
 $\mid (\lambda x \rightarrow (\lambda y \rightarrow e))$ -- A function that returns the identity function
 $\mid (e\ e)$ -- OR: a function that takes two arguments
-- and returns the second one!

How do I apply a function to two arguments?

- e.g. apply $(\lambda x \rightarrow (\lambda y \rightarrow y))$ to apple and banana?

$((\lambda x \rightarrow (\lambda y \rightarrow y))\ \text{apple})\ \text{banana}$ -- first apply to apple,
-- then apply the result to banana

Syntactic Sugar

instead of	we write
$\lambda x \rightarrow \lambda y \rightarrow (\lambda z \rightarrow e)$	$\lambda x \rightarrow \lambda y \rightarrow \lambda z \rightarrow e$
$\lambda x \rightarrow \lambda y \rightarrow \lambda z \rightarrow e$	$\lambda x\ y\ z \rightarrow e$
$((e_1\ e_2)\ e_3)\ e_4$	$e_1\ e_2\ e_3\ e_4$

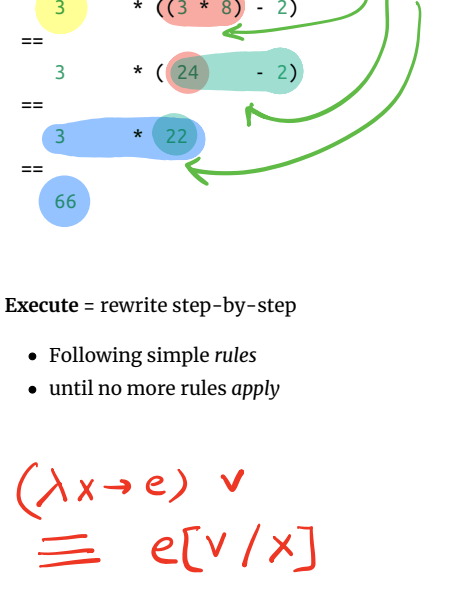
$\lambda x\ y \rightarrow y$ -- A function that takes two arguments
-- and returns the second one...

$(\lambda x\ y \rightarrow y)\ \text{apple}\ \text{banana}$ -- ... applied to two arguments

Semantics: What Programs Mean

How do I "run" / "execute" a λ -term?

Think of middle-school algebra:



Execute = rewrite step-by-step

- Following simple rules
- until no more rules apply

$(\lambda x \rightarrow e) \checkmark$
 $\equiv e[v/x]$

1/7/2025
wd1/22 OO-lambda

$e ::= x, y, z, \dots$
 $\mid (\lambda x \rightarrow e)$
 $\mid (e_1\ e_2)$

x is the **formal parameter**, e is the **body**

λ is the **function**, e_1 is the **argument**

$(\lambda x \rightarrow e)$ is the **function**, e_1 is the **argument**

Rewrite Rules of Lambda Calculus

1. β -step (aka function call)
2. α -step (aka renaming formal)

But first we have to talk about **scope**

$2 + 3 * 4$
 $\Rightarrow 2 + 12$
 $\Rightarrow 14$
 $\Rightarrow e$
 $\Rightarrow e_1$
 $\Rightarrow e_2$
 $\Rightarrow e_3$
 $\Rightarrow e_n$

$value$

Semantics: Scope of a Variable

The part of a program where a **variable** is **visible**

In the expression $(\lambda x \rightarrow e)$

- x is the newly introduced variable "binding site"
- e is the scope of x
- any occurrence of x in $(\lambda x \rightarrow e)$ is bound (by the binder λx)

For example, x is bound in:

$(\lambda x \rightarrow x)$ $(\lambda y \rightarrow x)$

$(\lambda x \rightarrow (\lambda y \rightarrow x))$

An occurrence of x in e is **free** if it's not bound by an enclosing abstraction

For example, x is free in:

$(x\ y)$ -- no binders at all!

$(\lambda y \rightarrow (x\ y))$ -- no λx binder

$((\lambda x \rightarrow (\lambda y \rightarrow y))\ x)$ -- x is outside the scope of the λx binder;
-- intuition: it's not "the same" x

③ Quiz

Is x bound or free in the expression $((\lambda x \rightarrow x) x)$?

A. first occurrence is bound, second is bound

B. first occurrence is bound, second is free

C. first occurrence is free, second is bound

D. first occurrence is free, second is free

$((\lambda x \rightarrow x) x)$
 $(\lambda x \rightarrow (x x))$

$10 - 5 - 5$
 $((10 - 5) - 5)$
 $(10 - (5 - 5))$

EXERCISE: Free Variables

An variable x is free in e if there exists a free occurrence of x in e

We can formally define the set of all free variables in a term like so:

$FV(x) = ???$
 $FV(\lambda x \rightarrow e) = ???$
 $FV(e1\ e2) = ???$

Closed Expressions

If e has no free variables it is said to be closed

- Closed expressions are also called combinators

What is the smallest closed expression?

x
 $x\ x$
 $x\ y$

$\lambda x \rightarrow x$

Semantics: Redex

A redex is a term of the form

$((\lambda x \rightarrow e1)\ e2)$
A function $(\lambda x \rightarrow e1)$

- x is the parameter
- $e1$ is the returned expression

Applied to an argument $e2$

- $e2$ is the argument

$2 + 2$

$\lambda x \rightarrow x\ x$

$(\text{apple} \rightarrow \text{dog})$

Semantics: β -Reduction

A redex β -steps to another term ...

$((\lambda x \rightarrow e1)\ e2) \Rightarrow e1[x := e2]$

where $e1[x := e2]$ means

$e1$ with all free occurrences of x replaced with $e2$

Computation by search-and-replace:

If you see an abstraction applied to an argument,

- In the body of the abstraction
- Replace all free occurrences of the formal by that argument

We say that $(\lambda x \rightarrow e1)\ e2$ β -steps to $e1[x := e2]$

Redex Examples

$((\lambda x \rightarrow x)\ \text{apple})$

$\Rightarrow$ apple

Is this right? Ask Elsa

QUIZ

$((\lambda x \rightarrow ((\lambda y \rightarrow y)\ x))\ \text{apple})$

$\Rightarrow$???

A. apple

B. $\lambda y \rightarrow \text{apple}$

C. $\lambda x \rightarrow \text{apple}$

D. $\lambda y \rightarrow y$

E. $\lambda x \rightarrow y$

QUIZ

$((\lambda x \rightarrow ((\lambda y \rightarrow y)\ x))\ \text{apple})$

$\Rightarrow$???

A. $((\text{apple}\ \text{apple})\ \text{apple})\ \text{apple}$

B. $((\lambda y\ \text{apple})\ y)\ \text{apple}$

C. $((\lambda y\ y)\ y)\ y$

D. apple

QUIZ

$((\lambda x \rightarrow ((\lambda y \rightarrow x))\ \text{apple})$

$\Rightarrow$???

A. $(\text{apple}\ (\lambda x \rightarrow x))$

B. $(\text{apple}\ (\lambda(\text{apple} \rightarrow \text{apple})))$

C. $(\text{apple}\ (\lambda x \rightarrow \text{apple}))$

D. apple

E. $(\lambda x \rightarrow x)$

EXERCISE

What is a λ -term fill_this_in such that

fill_this_in apple
 $\Rightarrow$ banana

ELSA: <https://goto.ucsd.edu/elsa/index.html>

Click here to try this exercise

$(\lambda x \rightarrow \text{banana})\ \text{apple}$
 $\Rightarrow$ banana

$(\lambda x \rightarrow x\ \text{banana})\ \text{apple}$
 $\Rightarrow$ banana

(a) $x\ \text{banana}$
(b) banana
(c) apple banana

A Tricky One

$((\lambda x \rightarrow ((\lambda y \rightarrow x))\ y)$

$\Rightarrow$ $\lambda y \rightarrow y$

Is this right?

$(\lambda x \rightarrow (\lambda m \rightarrow x))\ y$
 $\Rightarrow$ $(\lambda m \rightarrow y)$

Something is Fishy

$(\lambda x \rightarrow (\lambda y \rightarrow x))\ y$

$\Rightarrow$ $(\lambda y \rightarrow y)$

Is this right?

Problem: The free y in the argument has been captured by λy in body!

Solution: Ensure that *formals* in the body are different from *free-variables* of argument!

Capture-Avoiding Substitution

We have to fix our definition of β -reduction:

$(\lambda x \rightarrow e1)\ e2 \Rightarrow e1[x := e2]$

where $e1[x := e2]$ means $e1$ with all free occurrences of x replaced with $e2$

- $e1$ with all free occurrences of x replaced with $e2$
- as long as no free variables of $e2$ get captured

Formally:

$x[x := e] = e$

$y[x := e] = y$

-- as $x \neq y$

$(e1\ e2)[x := e] = (e1[x := e])\ (e2[x := e])$

$(\lambda x \rightarrow e1)[x := e] = (\lambda x \rightarrow e1)$

-- Q: Why leave 'e1' unchanged?

$(\lambda y \rightarrow e1)[x := e] = e1$
| not $(\lambda y \text{ in } FV(e)) = \lambda y \rightarrow e1[x := e]$

Oops, but what to do if y is in the free-variables of e ?

- i.e. if $\lambda y \rightarrow \dots$ may capture those free variables?

Rewrite Rules of Lambda Calculus

- β -step (aka function call)
- α -step (aka renaming formals)

Semantics: α -Renaming

Recall redex is a λ -term of the form

$((\lambda x \rightarrow e1)\ e2)$

A λ -term is in normal form if it contains no redexes.

Tricky Example Revisited

$((\lambda x \rightarrow (\lambda y \rightarrow x))\ y)$

-- rename 'y' to 'z' to avoid capture

$\Rightarrow$ $((\lambda x \rightarrow (\lambda z \rightarrow x))\ y)$

-- now do β -step without capture!

$\Rightarrow$ $(\lambda z \rightarrow y)$

To avoid getting confused,

- you can always rename formals,
- so different variables have different names!

Normal Forms

Recall redex is a λ -term of the form

$((\lambda x \rightarrow e1)\ e2)$

A λ -term is in normal form if it contains no redexes.


```
let TRIPLE = \x y z -> ???
let FST3 = \t -> ???
let SND3 = \t -> ???
let THD3 = \t -> ???

eval ex1:
  FST3 (TRIPLE apple banana orange)
=> apple

eval ex2:
  SND3 (TRIPLE apple banana orange)
=> banana

eval ex3:
  THD3 (TRIPLE apple banana orange)
=> orange
```

Programming in λ -calculus

- Booleans [done]
- Records (structs, tuples) [done]
- Numbers
- Lists
- Functions [we got those]
- Recursion

"3" $\lambda x \rightarrow f(f(f x))$
"1" $\lambda x \rightarrow f x$
"2" $\lambda x \rightarrow f(f x)$
"n" $\lambda x \rightarrow \underbrace{f \dots f}_n x$

λ -calculus: Numbers

Let's start with **natural numbers** (0, 1, 2, ...)

What do we *do* with natural numbers?

- Count: 0, inc
- Arithmetic: dec, +, -, *
- Comparisons: ==, <=, etc

Natural Numbers: API

We need to define:

- A family of **numerals**: ZERO, ONE, TWO, THREE, ...
- Arithmetic functions: **INC**, **DEC**, **ADD**, **SUB**, **MULT**
- Comparisons: **IS_ZERO**, **EQ**

Such that they respect all regular laws of arithmetic, e.g.

```
IS_ZERO ZERO ==> TRUE
IS_ZERO (INC ZERO) ==> FALSE
INC ONE ==> TWO
...
```

$INC = \lambda n \rightarrow \underline{\quad}??$

$\lambda x_1 x_2 \rightarrow x_1$

$\lambda x_1 x_2 \rightarrow x_2$

Natural Numbers: Implementation

Church numerals: a number n is encoded as a combinator that calls a function on an

argument n times

```
let ADD = \f x -> x
let ONE = \f x -> f x
let TWO = \f x -> f (f x)
let THREE = \f x -> f (f (f x))
let FOUR = \f x -> f (f (f (f x)))
let FIVE = \f x -> f (f (f (f (f x))))
let SIX = \f x -> f (f (f (f (f (f x))))))
...
```

$let\ INC = \lambda f\ x \rightarrow f \dots f(f\ x)$
 $(n\ f\ x) \Rightarrow \underbrace{f \dots f}_n (f(f\ x))$

QUIZ: Church Numerals

Which of these is a valid encoding of ZERO ?

- A: **let** ZERO = $\lambda f\ x \rightarrow x$
- B: **let** ZERO = $\lambda f\ x \rightarrow f$
- C: **let** ZERO = $\lambda f\ x \rightarrow f\ x$
- D: **let** ZERO = $\lambda x \rightarrow x$
- E: None of the above

Does this function look familiar?

$(n\ f\ x) \equiv \text{"call } f \text{ on } x \text{ } n \text{ times"}$

λ -calculus: Increment

-- Call 'f' on 'x' one more time than 'n' does

```
let INC = \n -> (\f x -> ???)
```

$INC\ 0 \Rightarrow N1$ $INC\ n = \lambda f\ x \rightarrow f(n\ f\ x)$
 $INC\ 1 \Rightarrow N2$
 $INC\ 2 \Rightarrow N3$

Example:

```
eval inc_zero :
  INC ZERO
=> (\n f x -> f (n f x)) ZERO
=> (\f x -> f (ZERO f x))
=> (\f x -> f x)
=> ONE
```

EXERCISE

Fill in the implementation of **ADD** so that you get the following behavior

[Click here to try this exercise](#)

```
let ZERO = \f x -> x
let ONE = \f x -> f x
let TWO = \f x -> f (f x)
let INC = \n f x -> f (n f x)
```

```
let ADD = fill_this_in
```

```
eval add_zero_zero:
  ADD ZERO ZERO ==> ZERO
```

```
eval add_zero_one:
  ADD ZERO ONE ==> ONE
```

```
eval add_zero_two:
  ADD ZERO TWO ==> TWO
```

```
eval add_one_zero:
  ADD ONE ZERO ==> ONE
```

```
eval add_one_one:
  ADD ONE ONE ==> TWO
```

```
eval add_two_zero:
  ADD TWO ZERO ==> TWO
```

QUIZ

How shall we implement **ADD** ?

- A. **let** ADD = $\lambda n\ m \rightarrow n\ INC\ m$
- B. **let** ADD = $\lambda n\ m \rightarrow INC\ n\ m$
- C. **let** ADD = $\lambda n\ m \rightarrow n\ INC$
- D. **let** ADD = $\lambda n\ m \rightarrow n\ (INC\ m)$
- E. **let** ADD = $\lambda n\ m \rightarrow n\ (INC\ n)$

Programming in λ -calculus

- Booleans [done]
- Records (structs, tuples) [done]
- Numbers [done]
- Lists
- Functions [we got those]
- Recursion

λ -calculus: Lists

Lets define an API to build lists in the λ -calculus.

An Empty List

NIL

Constructing a list

A list with 4 elements

```
CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL)))
```

Intuitively **CONS** h t creates a new list with

- head h
- tail t

Destructing a list

- HEAD** l returns the **first** element of the list
- TAIL** l returns the rest of the list

```
HEAD (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
=> apple
```

```
TAIL (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
=> CONS banana (CONS cantaloupe (CONS dragon NIL))
```

λ -calculus: Lists

```
let NIL = ???
let CONS = ???
let HEAD = ???
let TAIL = ???
```

```
eval exhd:
  HEAD (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
=> apple
```

```
eval extl:
  TAIL (CONS apple (CONS banana (CONS cantaloupe (CONS dragon NIL))))
=> CONS banana (CONS cantaloupe (CONS dragon NIL))
```

EXERCISE: Nth

Write an implementation of **GetNth** such that

- GetNth** n l returns the n-th element of the list l

Assume that l has n or more elements

```
let GetNth = ???
```

```
eval nth1:
  GetNth ZERO (CONS apple (CONS banana (CONS cantaloupe NIL)))
=> apple
```

```
eval nth1:
  GetNth ONE (CONS apple (CONS banana (CONS cantaloupe NIL)))
=> banana
```

```
eval nth2:
  GetNth TWO (CONS apple (CONS banana (CONS cantaloupe NIL)))
=> cantaloupe
```

[Click here to try this in Elsa](#)

QUIZ

You can write **SUM** using numerals but its tedious.

Is this a correct implementation of **SUM** ?

```
let SUM = \n -> ITE (ISZ n)
  ZERO
  (ADD n (SUM (DEC n)))
```

- A. Yes
- B. No

QUIZ

You can write **SUM** using numerals but its tedious.

Is this a correct implementation of **SUM** ?

```
let SUM = \n -> ITE (ISZ n)
  ZERO
  (ADD n (SUM (DEC n)))
```

- A. Yes
- B. No

No!

- Named terms in Elsa are just syntactic sugar
- To translate an Elsa term to λ -calculus: replace each name with its definition

```
\n -> ITE (ISZ n)
      ZERO
      (ADD n (SUM (DEC n))) -- But SUM is not yet defined!
```

Recursion:

- Inside *this* function
- Want to call the *same* function on DEC n

Looks like we can't do recursion!

- Requires being able to refer to functions *by name*,
- But λ -calculus functions are *anonymous*.

Right?

λ -calculus: Recursion

Think again!

Recursion:

Instead of

- ~~Inside this function I want to call the same function on DEC n~~

Lets try

- Inside *this* function I want to call *some* function `rec` on DEC n
- And BTW, I want `rec` to be the *same* function

Step 1: Pass in the function to call “recursively”

```
let STEP =
  \rec -> \n -> ITE (ISZ n)
            ZERO
            (ADD n (rec (DEC n))) -- Call some rec
```

Step 2: Do some magic to STEP, so `rec` is itself

```
\n -> ITE (ISZ n) ZERO (ADD n (rec (DEC n)))
```

That is, obtain a term **MAGIC** such that

```
MAGIC ==> STEP MAGIC
```

λ -calculus: Fixpoint Combinator

Wanted: a λ -term **FIX** such that

- **FIX STEP** calls **STEP** with **FIX STEP** as the first argument:

```
(FIX STEP) ==> STEP (FIX STEP)
```

(In math: a *fixpoint* of a function $f(x)$ is a point x , such that $f(x) = x$)

Once we have it, we can define:

```
let SUM = FIX STEP
```

Then by property of **FIX** we have:

```
SUM ==> FIX STEP ==> STEP (FIX STEP) ==> STEP SUM
```

and so now we compute:

```
eval sum_two:
  SUM TWO
==> STEP SUM TWO
==> ITE (ISZ TWO) ZERO (ADD TWO (SUM (DEC TWO)))
==> ADD TWO (SUM (DEC TWO))
==> ADD TWO (SUM ONE)
==> ADD TWO (STEP SUM ONE)
==> ADD TWO (ITE (ISZ ONE) ZERO (ADD ONE (SUM (DEC ONE))))
==> ADD TWO (ADD ONE (SUM (DEC ONE)))
==> ADD TWO (ADD ONE (SUM ZERO))
==> ADD TWO (ADD ONE (ITE (ISZ ZERO) ZERO (ADD ZERO (SUM DEC ZERO)))
==> ADD TWO (ADD ONE (ZERO))
==> THREE
```

How should we define **FIX**???

The Y combinator

Remember Ω ?

```
(\x -> x x) (\x -> x x)
==b> (\x -> x x) (\x -> x x)
```

This is *self-replicating code*! We need something like this but a bit more involved..

The Y combinator discovered by Haskell Curry:

```
let FIX = \stp -> (\x -> stp (x x)) (\x -> stp (x x))
```

How does it work?

```
eval fix_step:
  FIX STEP
==d> (\stp -> (\x -> stp (x x)) (\x -> stp (x x))) STEP
==b> (\x -> STEP (x x)) (\x -> STEP (x x))
==b> STEP ((\x -> STEP (x x)) (\x -> STEP (x x)))
--           ^^^^^^^^^^^ this is FIX STEP ^^^^^^^^^^^^^
```

That's all folks, Haskell Curry was very clever.

Next week: We'll look at the language named after him (**Haskell**)